

判断部分

1. 在遍历 ResultSet 结果集时，采用 `rs.getString(0)` 可获取首个字段的数据。 (✓)
2. 在 JDBC 中，可以使用 `PreparedStatement` 的 `execute()` 方法执行一个 sql 语句。 (✓)
3. 在 JDBC 中，可以使用 `PreparedStatement` 的 `execute` 方法获得一个 ResultSet 结果集。 (×)
4. 在 JDBC 中，可以使用 `while(rs.next()){...}` 方法遍历出 ResultSet 的所有记录。 (✓)
5. `font-type` 用来设置字体的类型。 (×)
6. 超链接只能在不同的网页之间进行跳转。 (×)
7. 在 `<form><form>` 标签对之间，不允许出现 `<p><ul>` 等非表单域元素。 (×)
8. 在 css 层叠规则中，`id 选择器` 样式表的优先级高于 `类选择器`。 (✓)
9. 超链接：是一种标记，形象的说法就是单击网页中的这个标记则能够加载另一个网页，这个标记可以做在本上也可以做在图像上。 (✓)
10. HTML 是 HyperText Markup Language（超文本标记语言）的缩写。超文本使网页之间具有跳转的能力，是一种信息组织的方式，使浏览者可以选择阅读的路径，从而可以不需要顺序阅读。 (✓)
11. 标识 `<b>` 无需 `</b>` 标识。 (×)
12. 只有 IE 浏览器支持 CSS，而其他浏览器不支持。 (×)
13. HTML 中，`空格` 的代码为 ` `。 (✓)
14. 外部 JS 脚本文件中必须包含 `<script>` 标签。 (×)
15. `<%! int c=2; out.print(c);%>` 符合 JSP 表达式。 (×)
16. JSP 页面中的 `指令标记`、`JSP 动作标记` 统称为 `脚本元素`。 (✓)
17. `Web 开发技术` 包括 `客户端` 和 `服务器端` 的技术。 (✓)
18. WEB-INF 目录中的文件能够通过浏览器访问。 (×)
19. Servlet 在 Servlet 容器中运行，其 `生命周期由容器管理`。 (✓)

填空部分

1. JDBC 连接的四个要素包括：[数据库用户名](#)、[数据库密码](#)、[jdbc 驱动](#)、[数据库 url](#)。
2. 在 JSP 页面中导入引用的 java 类或包，正确的方式是<@page import="java.util.*">。
3. Tomcat 的配置文件目录在[/conf](#)目录中。
4. 在 [HTML](#) 中有效、规范的注释声明是<!--这是注释-->。
5. 可插入多行注释的 [JavaScript](#) 语法是[/*This comment has more than one line*/](#)。
6. 在 [JavaScript](#) 中添加单行注释的语法是[//This is a comment](#)。
7. 在 HTML 中，以下关于 CSS 样式中文本及字体属性的一些说法：
 - ①[font-size](#) 用来设置文本字体的大小；
 - ②[font-weight](#) 用来设置字体的粗细；
 - ③[font-family](#) 用来设置字体的类型；
 - ④[text-align](#) 用来设置文本的对齐方式。
8. 在 HTML 中，["](#) 用来表示特殊字符引号。
9. 在 HTML 中，将表单中 INPUT 元素的 TYPE 属性值设置为 [Reset](#) 时，用于创建重置按钮。
10. 在 HTML 中，[colspan](#) 属性能够实现表格跨列；[rowspan](#) 属性能够实现表格跨行。
11. css 属性中用于指定内容与边框之间距离为 2px 的是 [padding:2px](#)。
12. 写"Hello World"的正确 Javascript 语法是 [document.write\("Hello World"\)](#)。
13. 在 JavaScript 中，有 2 种不同类型的循环，分别是 [for 循环](#)和 [while 循环](#)。
14. js 中使字符串中的字符变为小写的方法名是：[toLowerCase\(\)](#)。
15. 创建函数 [function myFunction\(\)](#)。
16. 在 html5 中，实现输入框提示占位符的属性是：[placeholder](#)。
17. 把 7.25 四舍五入为最接近的整数的方式是：[Math.round\(\)](#)。
18. 获得客户端浏览器的名称：[navigator.appName](#)。
19. JS 中关闭当前的窗口的方法是：[window.close\(\)/close\(\)/close](#)。
20. [JavaScript](#) 由 3 部分组成，分别是：[ECMAScript](#)，[BOM](#) 和 [DOM](#)。
21. 在警告框中写入"Hello World"：[alert\("Hello World"\)](#)。
22. 引用名为"xxx.js"的外部脚本的正确语法是：[<script src="xxx.js">](#)。
23. 当 [DOM](#) 加载完成后要执行的函数：[jQuery\(callback\)](#)。
24. 用来追加到指定元素的末尾：[appendTo\(\)](#)。
25. 有这样一个表单元素，想要找到这个 hidden 元素：[:hidden](#)。
26. 需要匹配包含文本的元素：[contains\(\)](#)。
27. 如果想要找到一个表格的指定行数的元素，用 [eq\(\)](#)方法可以快速找到指定元素。
28. 如果想在指定的元素后添加内容，[after\(content\)](#)是实现该功能的。
29. 在 jquery 中，如果想要从 DOM 中删除所有匹配的元素，可以用 [remove\(\)](#)。
30. 在 jquery 中，想要给第一个指定的元素添加样式，可以用 [css\(name\)](#)。
31. 在 jquery 中，如果想要获取当前窗口的宽度值，可以用 [width\(\)](#)。
32. 为每一个指定元素的指定事件（像 click）绑定一个事件处理器函数，可以用 [bind\(type\)](#)。
33. 在一个表单中，如果想要给输入框添加一个输入验证，可以用 [change\(fn\)](#)。
34. 当一个文本框中的内容被选中时，想要执行指定的方法时，可以使用 [select\(fn\)](#)。
35. 在 jquery 中想要实现通过远程 http [get](#) 请求载入信息功能的是[\\$.get\(url\)](#)事件。
36. 在 jquery 中指定一个类，如果存在就执行删除功能，如果不存在就执行添加功能，[toggleClass\(class\)](#)可以直接完成该功能。
37. 部署 WEB 项目时，应该部署项目到 Tomcat 的 [webapps](#) 目录下。
38. 在上传大文件时，form 表单元素采用 [POST](#) 方式提交请求。
39. 使用正则表达式声明 8 位数字 [var reg = ^d{8}\\$](#)。
40. foo 对象有 att 属性，获取 att 属性的值：[foo\["att"\]](#)。
41. 在 JSP 页面中，保存数据的范围由小到大依次是 [pageContext](#)，[request](#)，[session](#)，[application](#)。
42. [sessionScope](#) 适合保存用户登陆信息。

43. HttpServletResponse 对象中，用来把一个请求进行重定的方法是 sendRedirect()。
44. <c:choose>与<c:when>标签实现了 switch 功能。
45. Servlet 程序的每次调用的入口点是 service()。
46. form 表单的提交方式包括 GET 和 POST 方式。
47. 页面有元素, 可以通过 JQuery 的 \$('#name1')、\$('[name=name2]')、\$('input')获得。
48. 在 Servlet 中获得某请求参数的方法有 getParameter()和 getParameterValues()。
49. JSP 内置对象中，request 表示一次请求，response 表示一次响应，session 表示一次会话。

简答部分

1. 请详细描述 Servlet 的生命周期?

【答】

Servlet 有良好的生存期的定义，包括加载和实例化、初始化、处理请求以及服务结束。这个生存期由 `javax.servlet.Servlet` 接口的 `init`，`service` 和 `destroy` 方法表达。描述 Servlet 请求的流程。请 HTTP 协议基于请求响应模式，客户端向服务器发送一个请求，服务器则以一个状态行为作为响应。

①第 1 次访问 Servlet 时，实例化 Servlet 类并调用 `init` 方法进行初始化。

②每次访问 Servlet 时，调用 `service` 方法，`service` 方法根据请求的类型，再调用相应的 `doGet` 或 `doPost()` 方法。

③当 Servlet 容器关闭或重启时，调用 `destroy` 方法销毁 Servlet 对象。

④Servlet 默认是单实例的。

2. 雨课堂习题到此结束，剩下的靠自己了。

填空简答押题部分

1. HTML 基本结构: html, body, head

2. CSS 样式:

选择器: class 选择器, 类型选择器, id 选择器

定位样式: 绝对, 相对, 固定, 静态

绝对定位: position:absolute

相对定位: position:relative

固定定位: position:fixed

静态定位: position:static

3. JavaScript 和 jQuery:

①js 基本使用: 定义变量和函数: var a = 5;

②数组操作:

- 遍历 for

- 往数组最后添加一个元素 push()

- 往开头添加一个元素 unshift()

③常见的函数:

- json 的转换:

 - 字符串转对象: JSON.parse(text)

 - 对象转字符串: JSON.stringify(obj)

- 字符串转成数字: parseInt

- 定时器:

setTimeout()和 setInterval()这两个函数都有定时执行的作用, setTimeout 只执行一次, setInterval 会不断执行, 直到 clearInterval()停止。

setTimeout()和 setInterval()都是接受两个参数, 第一个参数是执行的函数, 第二个是延迟的毫秒数。

setTimeout()和 setInterval()都返回一个定时器 ID, 通过 clearTimeout()和 clearInterval()来消除定时器。

4. JavaScript 的异步编程:

①JavaScript 中的异步编程是指在代码执行过程中, 不需要等待某些操作完成就可以继续执行后续代码的一种编程方式, 在 JavaScript 中, 常见的异步编程方式包括回调函数、Promise 和 async/await。

②回调函数是一种最基础的异步编程方式, 它通过将需要异步执行的代码封装在一个函数中, 并在操作完成后调用该函数来实现异步执行。例如, 可以在 setTimeout()函数中传递一个回调函数, 在延迟指定时间后执行该回调函数。

③Promise 是一种更高级的异步编程方式, 它可以更好地处理异步操作的结果和错误。Promise 可以将异步操作封装成一个 Promise 对象, 并使用.then()和.catch()方法处理异步操作的结果和错误。

④async/await 是 ES2017 中引入的一种异步编程方式, 它可以更直观地编写异步代码。使用 async/await 可以将异步操作看作同步操作, 通过 async 函数定义异步函数, 使用 await 关键字等待异步操作完成并返回结果。

异步编程可以提高代码的性能和可维护性, 避免了阻塞线程等问题, 因此在 JavaScript 中被广泛应用。

5. Servlet:

①Servlet 的配置: web.xml 的方式和注解方式。

②生命周期: 第一次访问的时候被创建, 服务器 (Servlet 容器) 关闭的时候销毁。

③Servlet 是线程不安全的, 专业上称为非线程安全。共享数据的时候, 可以写同步方法来实现线程安全。

④两个核心的接口: HttpServletRequest 和 HttpServletResponse 对象, 分别处理“客户端发起的请求”和“服务器端的响应”。

- request 可以获取请求参数

- request 可以获取请求头信息

- request 可以设置属性

- request 可以转发请求

⑤转发和重定向:

- request.getRequestDispatcher("/show.jsp").forward(request,response);

•response.sendRedirect("/show.jsp");

6. MVC 模式:

①MVC 是一个常见的软件开发模式（设计模式），分为模型（Model）、视图（View）、控制器（Controller），降低程序的耦合度，提高内聚性和扩展性。

②模型表示业务数据和业务逻辑。如实体类、服务层、dao 层等代码模块。

③视图表示用户界面。

④控制层协调模型和视图之间的交互，用户操作转换成模型的操作，并将模型展示到视图中。

⑤MVC 的优点：采用 MVC 模式有助于代码的可复用性，开发人员更容易合作，修改、测试等好处。提高开发效率。

7. jsp 内置对象及其作用（9 个）：

①request: 表示 http 请求对象，获取请求参数和访问请求属性。

②response: 表示 http 响应，提供响应状态和头信息。

③pageContext: jsp 页面上下文，获取请求，响应、会话和 application 对象。

④application: （应用上下文）表示应用程序对象，对应 ServletContext。

⑤session: 表示 HttpSession（会话）对象，可以访问会话的属性。

⑥out: 表示响应的输出流，可以直接写入内容。

⑦config: 提供 jsp 配置信息。

⑧page: 表示当前 JSP 页面的访问。

⑨exception: 异常对象，只能在错误页面中使用，访问异常信息。

8. EL 表达式:

①EL 表达式中既可以调用方法，又可以通过 param 内置对象来直接获取参数。

②在 jsp 中，以下三种代码是同一个效果：

<% out.print(user.getName());%> 直接 out 输出

<%= user.getName()%> 表达式输出

\${user.name} EL 表达式

③EL 表达式也有隐藏的内置对象

EL 表达式的内置对象和 JSP 的内置对象只有一个是相同的 pageContext

程序押题部分

1. 第一道大题:

```
<!DOCTYPE html>

<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
  <div align="center">
    <h1 style="color: red;">课程信息录入</h1>
    <a href="index.jsp">返回主页</a>
    <form action="##" method="post" onsubmit="return check()">
      <div>
        课程名称<input type="text" id="name" name="name"/>
      </div>
      <div>
        任课教师<input type="text" id="teacher" name="teacher" />
      </div>
      <div>
        上课地点<input type="text" id="classroom" name="classroom" />
      </div>
      <div>
        <button type="submit">保 存</button>
      </div>
    </form>
  </div>
  <script type="text/javascript">
    function check() {
      var name = document.getElementById("name");
      var teacher = document.getElementById("teacher");
      var classroom = document.getElementById("classroom");

      //非空
      if(name.value == '') {
        alert('课程名称为空');
        name.focus();
        return false;
      }
      //教师
      if(!/^[a-zA-Z]+$/ .test(teacher.value)){
        alert('教师名称只能输入大小写');
        return false;
      }
      if(Number(classroom.value)<5){
        alert('上课名称错误');
        classroom.focus();
        return false;
      }
    }
  </script>
</body>
</html>
```

```
}
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```


2. 第二道大题:

①从数据库中提取数据

```
public List<Course> list() {
    String sql = "select * from course";
    List<Course> list = new ArrayList<>();
    Connection conn = DBUtil.getConn();
    Statement state = null;
    ResultSet rs = null;

    state = conn.createStatement();
    rs = state.executeQuery(sql);
    Course bean = null;
    while (rs.next()) {
        int id = rs.getInt("id");
        String name2 = rs.getString("name");
        String teacher2 = rs.getString("teacher");
        String classroom2 = rs.getString("classroom");
        bean = new Course(id, name2, teacher2, classroom2);
        list.add(bean);
    }
    DBUtil.close(rs, state, conn);

    return list;
}
```

②用 servlet 将提取的数据显示在表单中

```
@WebServlet("/CourseServlet")
public class CourseServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    public CourseServlet() {
        super();
        // TODO Auto-generated constructor stub
    }

    CourseDao dao = new CourseDao();

    private void list(HttpServletRequest req, HttpServletResponse resp) throws IOException, ServletException{
        req.setCharacterEncoding("utf-8");
        List<Course> courses = dao.list();
        req.setAttribute("courses", courses);
        req.getRequestDispatcher("list.jsp").forward(req, resp);
    }
}
```

③在表单中进行显示

```
<!DOCTYPE html>

<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<style>
```

```
.tb, td {
    border: 1px solid black;
    font-size: 22px;
}
</style>
</head>
<body>
    <div align="center">
        <h1 style="color: red;">课程信息列表</h1>
        <a href="index.jsp">返回主页</a>
        <table class="tb">
            <tr>
                <td>id</td>
                <td>课程名称</td>
                <td>任课教师</td>
                <td>上课地点</td>
            </tr>
            <c:forEach items="${courses}" var="item">
                <tr>
                    <td>${item.id}</td>
                    <td>${item.name}</td>
                    <td>${item.teacher}</td>
                    <td>${item.classroom}</td>
                </tr>
            </c:forEach>
        </table>
    </div>
</body>
</html>
```

补充部分

1. 修改数据:

```
//第一步: 创建 queryRunner 对象, 用来操作 sql 语句
QueryRunner qr = new QueryRunner(JDBCUtils.getDataSource());

//第二步: 创建 sql 语句
String sql = "update user set name = ? where id = ?";

//第三步: 执行 sql 语句,params:是 sql 语句的参数。注意, 给 sql 语句设置参数的时候, 按照 user 表中字段的顺序
try {
    int update = qr.update(sql, "张飞",7);
    System.out.println(update);
} catch (SQLException e) {
    e.printStackTrace();
}
```

2. 修改数据:

```
//第一步: 创建 queryRunner 对象, 用来操作 sql 语句
QueryRunner qr = new QueryRunner(JDBCUtils.getDataSource());

//第二步: 创建 sql 语句
String sql = "update user set name = ? where id = ?";

//第三步: 执行 sql 语句,params:是 sql 语句的参数。注意, 给 sql 语句设置参数的时候, 按照 user 表中字段的顺序
try {
    int update = qr.update(sql, "张飞",7);
    System.out.println(update);
} catch (SQLException e) {
    e.printStackTrace();
}
```

3. 删除数据:

```
//第一步: 创建 queryRunner 对象, 用来操作 sql 语句
QueryRunner qr = new QueryRunner(JDBCUtils.getDataSource());

//第二步: 创建 sql 语句
String sql = "delete from user where id = ?";

//第三步: 执行 sql 语句,params:是 sql 语句的参数, 给 sql 语句设置参数的时候, 按照 user 表中字段的顺序
try {
    int update = qr.update(sql, 7);
    System.out.println(update);
} catch (SQLException e) {
    e.printStackTrace();
}
}
```

4. 假设有一个过滤器, 其全限定类名为: com.web.EncodingFilter, 要求它能拦截所有的请求, 且传入字符集参数

charset。请在 web.xml 文件中注册该过滤器。

```
<filter>
    <filter-name>EncodingFilter</filter-name>
<filter-class>com.web.EncodingFilter
</filter-class>
<init-param>
    <param-name>charset</param-name>
    <param-value>UTF-8</param-value>
</init-param>
</filter>
<filter-mapping>
    <filter-name>EncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

5. 在 JavaScript 中，对数组[6, 3, 1, 5, 2, 4]按数值进行从大到小排序，并将结果以分号相隔拼成一个字符串，并使用 alert 弹出该字符串。

```
<script>
var arr = [6, 3, 1, 5, 2, 4];
arr.sort(function(x, y) {
    return y-x;
});
var str = arr.join(';');
alert(str);
</script>
```

6. 在 JavaScript 中，编写一个用户(User)类，属性有名字(name)、年龄(age)，方法有 show()，在方法中使用 alert 弹出用户的名字和年龄。定义一个对象并调用方法。

```
<script>
    function User(name, age) {
        this.name = name;
        this.age = age;
    }
    User.prototype.show = function() {
        alert(this.name + “,” + this.age);
    }
    var user = new User('test', 20); //1 分
    user.show();
</script>
```